# The Why, What, and How of Containerisation

Containerisation has revolutionised software development by offering lightweight, portable, and scalable solutions for application deployment. Containers empower teams to deploy applications faster and more reliably in diverse environments. Let's find out how and why they work so well.

In the modern era of software development, agility, scalability, and portability have become essential for building and deploying applications. Enter containerisation—a technology that has revolutionised the way developers package, deploy, and manage applications across various environments. From microservices architecture to DevOps pipelines, containerisation is now a cornerstone of cloud-native development.

## Why containerisation?

The need for containerisation arises from the challenges developers and operations teams face when building, deploying, and maintaining software in diverse environments. Let's dive deeper into the reasons.

***The "It works on my machine" problem:*** This age-old problem arises when code that runs perfectly on a developer's local system fails in staging or production environments. This discrepancy is usually due to differences in:

- Operating system versions (e.g., Ubuntu 18.04 vs 22.04)
- Missing or different versions of dependencies (e.g., Python 3.6 vs 3.10)
- System libraries or tools
- Environment variables and configuration files

Containers fix this by encapsulating the application along with all its dependencies, libraries, and environment settings into a single, consistent unit. As a result, the containerised application behaves the same way—regardless of where it is deployed, be it a developer's laptop, a test server, or a public cloud.

*Example:* A Flask web app containerised with Python 3.9 and specific libraries will run identically on all systems with Docker installed.

***Slow and inefficient deployment with VMs:*** Traditional virtual machines (VMs) emulate entire operating systems. While VMs provide good isolation, they are resource-intensive:

- Require gigabytes of storage per instance
- Take minutes to boot
- Need separate OS licences and patching

Containers, on the other hand, share the host OS kernel